

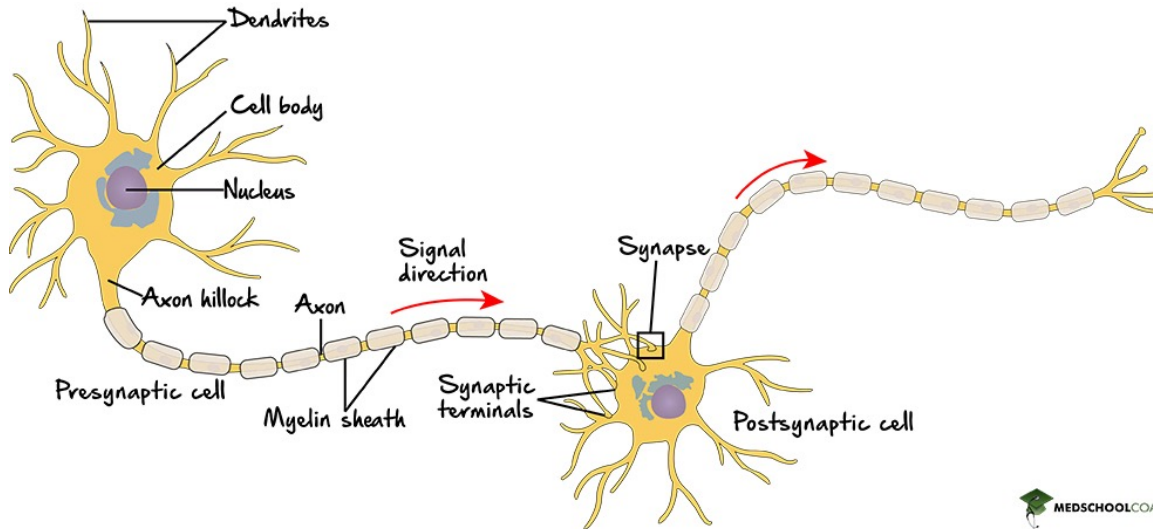
Brief History of Artificial Intelligence



A lecture by Maneesha Rajapaksha

🧠 The McCulloch-Pitts Neuron (1943)

The foundational idea: a brain cell is just a logic gate. If it gets enough electrical signals at once, it fires. Otherwise, it stays silent. McCulloch and Pitts asked — *can we write that as math?*



Warren McCulloch (neurophysiologist) and Walter Pitts (logician) published "*A Logical Calculus of Ideas Immanent in Nervous Activity*" — the first mathematical model of how neurons fire. They showed that neural networks could, in principle, compute any logical function. This gave scientists a blueprint: **the brain is computable.**

The Mathematical Model

The MP neuron is the simplest possible mathematical abstraction of a biological neuron. It has three parts:

- 1. Binary Inputs** — each input x_i is either 0 (off) or 1 (on), just like a nerve impulse either arrives or doesn't.
- 2. Weights** — each input has an associated weight w_i . In the original paper these were all 1, but the weight can be negative to model **inhibitory** synapses (signals that *prevent* firing).
- 3. Threshold activation** — the neuron computes the weighted sum and compares it to a threshold θ :

output = 1 if $\sum(w_i \cdot x_i) \geq \theta$

output = 0 if $\sum(w_i \cdot x_i) < \theta$

That's it. **No learning, no gradients, no probabilities** — just a sum and a hard cutoff.



Implementing Logic Gates

The power of this model was that it could compute **any Boolean logic function** by choosing the right weights and threshold.

Gate	Inputs	Weights	Threshold θ	How it works
AND	x_1, x_2	1, 1	2	Both must fire $\rightarrow$ $\text{sum}=2 \geq 2 \checkmark$
OR	x_1, x_2	1, 1	1	Either fires $\rightarrow$ $\text{sum} \geq 1 \checkmark$
NOT	x_1	-1	0	Input off $\rightarrow \text{sum}=0 \geq$ 0 fires; input on $\rightarrow$ $\text{sum}=-1 < 0$ silent
NOR	x_1, x_2	-1, -1	0	Only fires if both inputs are 0



Biological Analogy

Biology

McCulloch-Pitts Model

Dendrites receiving signals

Inputs $x_1, x_2, \dots, x_n$

Synaptic strength

Weights w_i

Inhibitory synapse

Negative weight $w_i < 0$

Axon hillock summation

$\sum (w_i \cdot x_i)$

Action potential threshold

Threshold θ

Neuron fires / doesn't fire

Output = 1 or = 0

The Notation System

$N_i(t)$ = neuron i fires at time t (1 = fires, 0 = silent)

$.=.$ = "if and only if" / logical equivalence

$\vee$ = OR

$\cdot$ (dot) = AND

$-$ = NOT (inhibition)

(Ex) = "there exists an x such that..." (existential quantifier)

$t-1, t-2$ = synaptic delay — signal takes 1 or more time steps to arrive

Eg- $N_1(t) . = . N_1(t-1)$

N_1 fires at time t IF N_1 fired at time $t-1$

→ **Self-excitation / memory.** The neuron keeps itself active once triggered. Models a persistent firing loop — the simplest form of neural memory.

Eg - $N_2(t) . = . N_1(t-1) \vee N_3(t-1)$

N_2 fires IF N_1 fired OR N_3 fired (one step ago)

→ **OR gate.** Any one excitatory input is sufficient to trigger the neuron. This is the biological equivalent of convergent excitation — multiple pathways activating the same cell.

Eg- $N_3(t) \models N_1(t-1) \wedge N_2(t-1)$

N_3 fires IF N_1 fired AND N_2 fired (one step ago)

→ **AND gate**. Both inputs must arrive simultaneously. Models coincidence detection — a fundamental operation in sensory processing (e.g. detecting two features co-occurring).

⚠ Critical Limitations

The MP neuron was revolutionary but had major constraints that defined the next 80 years of AI research:

No learning — weights and threshold are set by hand. The neuron cannot adjust itself from experience. This was the core problem Frank Rosenblatt solved 15 years later with the **Perceptron**.

Binary only — inputs and outputs are strictly 0 or 1. No continuous values, no probabilities, no uncertainty.

No XOR — a single MP neuron cannot compute XOR (exclusive or), because XOR is not *linearly separable*. This limitation — later formalized by Minsky & Papert in 1969 — killed neural network research for a decade.

No time — the model is stateless. There is no memory of past activations, no recurrence.

Alan Turing's "Computing Machinery and Intelligence" — 1950

Turing asked the now-legendary question: "*Can machines think?*" He proposed the **Imitation Game** (the Turing Test) — if a machine could converse indistinguishably from a human, it could be considered intelligent. This paper framed the philosophical debate that still continues today.

The Imitation Game

Turing's original setup had 3 participants:

Participant	Role
Interrogator (C)	A human judge in a separate room, communicating only via text
Human (B)	A human in another room, trying to help the judge identify them correctly
Machine (A)	A computer, trying to convince the judge it is the human

All communication is through a **teletype** (typewritten text) — removing voice, appearance, and all physical cues. The interrogator asks any questions they like. If the interrogator cannot reliably distinguish the machine from the human, the machine is said to have passed.



Has Any AI Passed It?

Eugene Goostman (2014) — A chatbot posing as a 13-year-old Ukrainian boy reportedly convinced 33% of judges at a University of Reading competition. Widely disputed — the persona gave the chatbot a huge excuse for odd answers.

Large Language Models (2020s) — GPT-4, Claude, and similar models routinely fool people in text conversations. In informal settings, the Turing Test is essentially "solved" for short interactions. But researchers argue this reveals the test's inadequacy, not machine consciousness.



ELIZA — The World's First Chatbot (1966)

ELIZA is extraordinarily simple by modern standards. It contains **no knowledge, no understanding, no model of the world** — only a pattern-matching engine operating on text.

Pattern: "I am (.*)"

Response: "Why do you say you are [captured group]?"

Input: "I am feeling hopeless"

Captured: "feeling hopeless"

Output: "Why do you say you are feeling hopeless?"

YOU: I feel sad

ELIZA: Why do you feel sad?

YOU: My brother is annoying

ELIZA: Tell me more about your brother.

YOU: I am tired

ELIZA: How long have you been tired?

The 1956 Dartmouth Conference

Summer of 1956 — roughly 6–8 weeks — at **Dartmouth College**, Hanover, New Hampshire. Organised by four researchers who submitted a proposal to the Rockefeller Foundation requesting funding for a two-month study. The proposal contained the first ever formal use of the phrase "**artificial intelligence**"

Before it, scattered researchers worked in isolation. After it, artificial intelligence existed as a named, funded, organised field.

Person	Affiliation	Legacy
John McCarthy	Dartmouth College	Coined "AI", invented LISP, Turing Award 1971
Marvin Minsky	Harvard	MIT AI Lab co-founder, Turing Award 1969
Claude Shannon	Bell Labs	Father of information theory
Nathaniel Rochester	IBM	Designed the IBM 701 computer

The Perceptron (1957–1958)

The Perceptron was the first machine that could genuinely learn from experience — not just execute a program, but adjust its own internal parameters based on examples. This was a conceptual earthquake. It meant intelligence wasn't something you had to hand-code; it could *emerge* from training.

Frank Rosenblatt

Frank Rosenblatt was a Cornell University psychologist and computer scientist. He was fascinated not by formal logic or theorem-proving — the dominant AI paradigm post-Dartmouth — but by the brain itself. His approach was explicitly **biological**: build something that *learns the way neurons learn*.

In 1957–1958 he designed and implemented the Perceptron, first as a software simulation and then as **Mark I** — a custom-built hardware machine at Cornell Aeronautical Laboratory, funded by the US Navy. It had 400 photocells as inputs and could learn to recognize visual patterns.

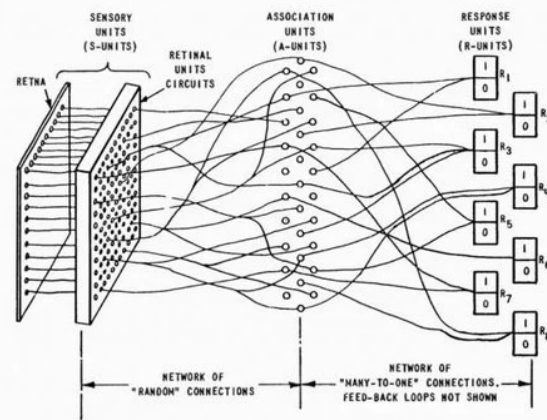
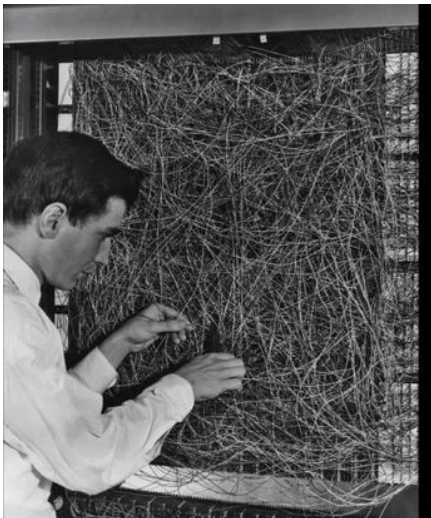
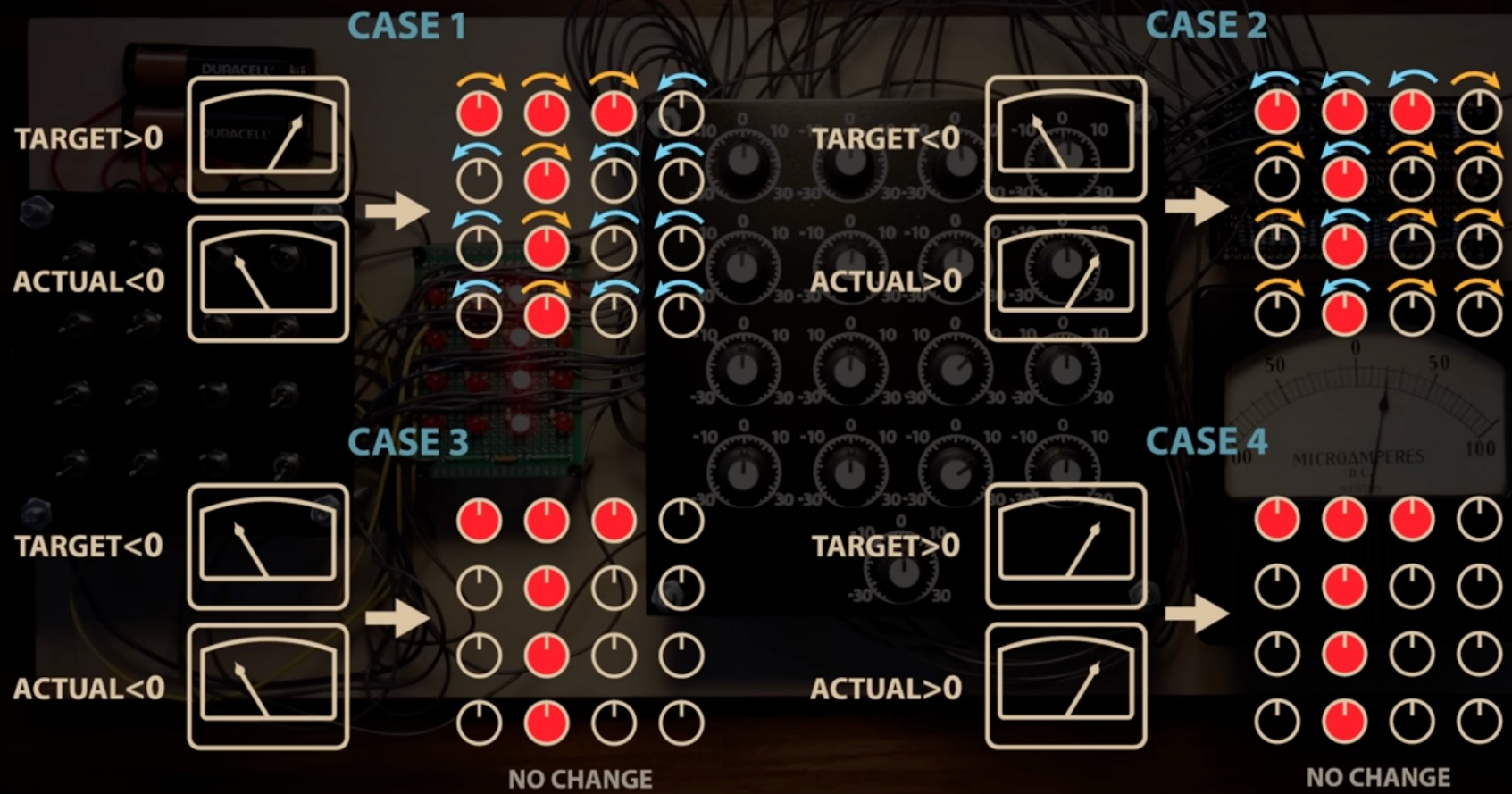


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

LEARNING ALGORITHM



Backpropagation

Backpropagation is the algorithm that made deep learning possible. It solves the fundamental problem of multi-layer networks: given that the output is wrong, *which weights are to blame, and by how much?* The answer comes from calculus — specifically, the chain rule applied in reverse through the network.

1960	Henry Kelley / Arthur Bryson	Backprop-like ideas in optimal control theory
1970	Seppo Linnainmaa	Published automatic differentiation — the mathematical core — in his Master's thesis
1974	Paul Werbos	Described backprop for neural networks in his Harvard PhD thesis — largely ignored
1982	David Parker	Rediscovered it independently
1986	Rumelhart, Hinton & Williams	Published " <i>Learning Representations by Back-propagating Errors</i> " in Nature — the paper that changed everything

The fundamental problem backprop solves is called the **credit assignment problem**: in a network with millions of weights and a single scalar loss, how do you know which weight caused the error? The answer is calculus. Every weight's contribution to the loss can be computed exactly using the **chain rule**:

$$\partial L / \partial w = \partial L / \partial a_{\text{out}} \times \partial a_{\text{out}} / \partial z_{\text{out}} \times \partial z_{\text{out}} / \partial w$$

Step 1: Forward Pass

Compute activations layer by layer, left to right. Store every intermediate value — you'll need them for the backward pass.

Step 2: Compute Output Gradient

How does the loss change with respect to the output?

Step 3: Propagate Backward — Chain Rule

Work right to left. At each layer, multiply the incoming gradient by the local derivative

Step 4: Update Weights — Gradient Descent

Nudge every weight in the direction that reduces loss

Rule Based Expert Systems Boom

DENDRAL (1965–80) — determined molecular structures from mass spectrometry data. Used for 15 years in real labs — the first proof expert systems could work outside academia.

MYCIN (1972) — diagnosed bacterial infections using 600 IF-THEN rules with certainty factors. Outperformed junior doctors (65% correct vs 42–62%). Never deployed due to liability concerns — the first AI ethics debate.

XCON / R1 (1980) — configured VAX computer orders for DEC using 2,500 rules. Saved \$40 million per year. Triggered a global expert systems gold rush — billions invested, hundreds of commercial deployments.

Deep Blue (1997) - Deep Blue was IBM's chess-playing computer that defeated world champion Garry Kasparov in May 1997 — the first time a machine beat a reigning world champion in a classical chess match. Not neural networks, not learning — pure brute-force search with hand-crafted evaluation.

Four fatal flaws destroyed the expert systems industry simultaneously:

Brittleness — systems performed perfectly within their domain and catastrophically outside it. Real problems don't stay in domains.

Knowledge acquisition bottleneck — extracting rules from human experts was slow, expensive, and often impossible. Experts couldn't articulate their own tacit knowledge.

Maintenance nightmare — 2,500 rules interact in unpredictable ways. Fixing one rule broke five others. XCON needed a full-time team just to keep it running.

No learning — every piece of knowledge had to be manually encoded. Systems couldn't improve from experience or adapt to new situations.

The AlphaZero verdict (2017): DeepMind's AlphaZero learned chess from scratch with no rules except legal moves, via pure self-play reinforcement learning, in **4 hours** — then defeated the strongest rule-based successor to Deep Blue convincingly. What took IBM years of knowledge engineering, AlphaZero learned from data alone.

AI Renaissance

1. Data Explosion (Internet Effect)

The internet unintentionally created the **largest training dataset in history**:

- **Text (1990s)**: Massive web pages enabled early NLP and search engines like Google.
- **Images (2000s)**: Platforms like Flickr and Facebook generated billions of labeled images.
- **ImageNet (2009)**: Built by Fei-Fei Li, it became the key benchmark that triggered deep learning progress.
- **Behavioral Data**: Clicks, searches, and purchases (Google, Amazon, Netflix) became training signals.

Result: ML moved from **data scarcity** → **data abundance**



2. Compute Revolution

Training large models became feasible due to:

- **Moore's Law**: Exponential CPU improvements
- **GPUs + CUDA**: Parallel computation made neural networks practical (via NVIDIA)
- **Cloud Computing**: Amazon Web Services and others made compute accessible

Result: Training went from **weeks** → **days** → **hours**

3. Algorithm Advances

Key improvements made learning actually work:

- **SVMs:** Dominated pre-2012 ML
- **Deep Belief Networks (2006):** Led by Geoffrey Hinton, solved training issues in deep nets
- **CNNs:** Introduced by Yann LeCun, became practical with GPUs

Result: Deep learning became **trainable and scalable**

4. The Turning Point — AlexNet (2012)

Built by Alex Krizhevsky, Ilya Sutskever, and Hinton:

- Crushed previous benchmarks on ImageNet
- Used GPUs + deep CNNs
- Marked the **start of the deep learning era**



What It Led To

AlexNet (2012) → GANs (2014) → ResNet (2015) → Transformer (2017) → BERT/GPT-1 (2018) → GPT-3 (2020) → ChatGPT (2022). The throughline is unbroken — internet data, GPU compute, and revived algorithms all converging at ImageNet was the starting gun for modern AI.